Polymorphism in Java

Polymorphism is one of the four pillars of Object-Oriented Programming (OOP), along with Encapsulation, Inheritance, and Abstraction. Derived from the Greek words "poly" (many) and "morph" (forms), **polymorphism** allows one entity (method, object, or operator) to take multiple forms.

In Java, polymorphism enables flexibility and reusability by allowing the same method or operator to perform different tasks depending on the context.

Types of Polymorphism in Java

1. **Compile-Time Polymorphism (Static Binding)**:
   - Achieved through **method overloading**.
   - The method to be called is determined at compile time.
2. **Run-Time Polymorphism (Dynamic Binding)**:
   - Achieved through **method overriding**.
   - The method to be called is determined during program execution.

---

1. Compile-Time Polymorphism (Method Overloading)

Method overloading occurs when multiple methods in the same class share the same name but differ in:

- The number of parameters.
- The type of parameters.
- The order of parameters.

Key Points:

- Method overloading cannot be achieved by changing the return type alone.
- Overloading improves code readability.

Example:

```
class Calculator {
    // Method to add two integers
    int add(int a, int b) {
        return a + b;
```

```
    }

    // Overloaded method to add three integers
    int add(int a, int b, int c) {
        return a + b + c;
    }

    // Overloaded method to add two doubles
    double add(double a, double b) {
        return a + b;
    }
}

public class CompileTimePolymorphismExample {
    public static void main(String[] args) {
        Calculator calc = new Calculator();

        System.out.println("Sum of two integers: " + calc.add(2, 3));
// Calls add(int, int)
        System.out.println("Sum of three integers: " + calc.add(2, 3, 4));
// Calls add(int, int, int)
        System.out.println("Sum of two doubles: " + calc.add(2.5, 3.5));
// Calls add(double, double)
    }
}
```

Output:

```
Sum of two integers: 5
Sum of three integers: 9
Sum of two doubles: 6.0
```

---

2. Run-Time Polymorphism (Method Overriding)

Method overriding occurs when a subclass provides its own implementation of a method already defined in its superclass.

Key Points:

- The method in the subclass must have the same name, return type, and parameters as the method in the superclass.
- `@Override` annotation is used to indicate overriding.
- The overridden method is called based on the actual object, not the reference type.

Example:

```java
class Animal {
    void sound() {
        System.out.println("Some generic animal sound");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}

class Cat extends Animal {
    @Override
    void sound() {
        System.out.println("Cat meows");
    }
}

public class RunTimePolymorphismExample {
    public static void main(String[] args) {
        Animal myAnimal;  // Reference of type Animal

        myAnimal = new Dog();  // Object of type Dog
        myAnimal.sound();      // Calls Dog's sound method

        myAnimal = new Cat();  // Object of type Cat
        myAnimal.sound();      // Calls Cat's sound method
    }
}
```

Output:

```
Dog barks
Cat meows
```

Dynamic Method Dispatch

Dynamic method dispatch is a mechanism by which a call to an overridden method is resolved at runtime. It is the foundation of run-time polymorphism in Java.

Example:

```java
class Parent {
    void display() {
        System.out.println("Parent class display method");
    }
}

class Child extends Parent {
    @Override
    void display() {
        System.out.println("Child class display method");
    }
}

public class DynamicDispatchExample {
    public static void main(String[] args) {
        Parent obj = new Child();  // Parent reference, Child object
        obj.display();             // Calls Child's display method
    }
}
```

Output:

```
Child class display method
```

Advantages of Polymorphism

1. **Code Reusability**:
   - Common behavior can be reused across different classes.
2. **Flexibility**:
   - Methods can be modified in the subclass without changing the parent class.
3. **Extensibility**:
   - New implementations can be added without affecting existing code.
4. **Maintainability**:
   - Centralized code for shared behaviors makes maintenance easier.
5. **Readability**:
   - Overloading and overriding make code intuitive.

Disadvantages of Polymorphism

1. **Performance Overhead**:

- Run-time polymorphism may introduce slight performance degradation due to dynamic method resolution.

2. **Complex Debugging**:
   - Tracing overridden methods in large hierarchies can be challenging.
3. **Tight Coupling**:
   - Extensive use of polymorphism can lead to tightly coupled systems.

Key Differences Between Overloading and Overriding

Aspect	Overloading	Overriding
Binding	Compile-time (Static)	Run-time (Dynamic)
Classes Involved	Same class	Parent and child classes
Method Signature	Different parameters (number/type/order)	Same name, return type, and parameters
Keyword	No specific keyword	`@Override` (optional, but recommended)